

Geometry Streaming Server Product Guide API

Contents

REST API Manual	1
Create Project: POST /api/projects/ <i>name</i>	1
Upload Data: POST /api/projects/ <i>name</i> /upload	2
Run Project: POST /api/projects/ <i>name</i> /run	4
Get Project Metadata: GET /api/projects/ <i>name</i>	5
List Projects: GET /api/projects	6
Get Project Result URL: GET /api/projects/ <i>name</i> /result	7
View Project: GET /api/projects/ <i>name</i> /view	7
Delete Project: DELETE /api/projects/ <i>name</i>	8
Task API	8
Get Master Task ID: GET /api/tasks/master/id	10
Get Task Metadata: GET /api/tasks/ <i>id</i>	10
Get Task Log: GET /api/tasks/ <i>id</i> /log	11

REST API Manual

All API methods require an API token to be set as an **x-landform-token** HTTP header or **landform-token** cookie.

Unless otherwise specified, all API arguments may be specified as either URL query parameters or as fields in a **application/json** or **application/x-www-form-urlencoded** HTTP request body. If both the query parameter and body field are present the former takes precedence.

Create Project: POST /api/projects/*name*

Create the named project. Implements the task API.

Accepts the following arguments:

- *tilingscheme*: tiling scheme; one of **Bin**, **QuadX**, **QuadY**, **QuadZ**, **Oct**, or **UserDefined**; default **Bin**
- *skirtmode*: skirt mode; one of **X**, **Y**, **Z**, **None**, **Normal**; default **None**
- *reconmethod*: reconstruction method; one of **Poisson**, **FSSR**; default **Poisson**

- *facespertile*: target maximum faces per tile; default 2000
- *tileresolution*: maximum image resolution per tile; default 256
- *projecttype*: project type; currently only **GenericTiling** is supported
- *exportmeshformat*: additional mesh format to write, one of **obj**, **ply**, **stl**, or none; default none
- *exportimageformat*: additional image format to write, one of **tif**, **png**, **jpg**, or none; default none
- *maxleafgroupsize*: maximum number of leaves to process as a group; default 32

Fails with HTTP status 400 (bad request) if a project with the same name already exists.

If multiple calls to this API are made in rapid succession with the same project name only one will succeed. However, in this case it is possible that the HTTP status of more than one of the calls may be HTTP 200 (OK).

The return status may be HTTP 200 even if one or more arguments are invalid. However the project will not actually be created in that case.

Example: create project “testproj”

Request

```
POST /api/projects/testproj HTTP/1.1
Host: https://landform.hi.jpl.nasa.gov
x-landform-token: API_TOKEN
Content-Type: multipart/form-data; boundary=----WebKitFormBoundary7MA4YWxkTrZu0gW

-----WebKitFormBoundary7MA4YWxkTrZu0gW--
```

Response

```
HTTP/1.1 200
status: 200
content-type: application/json; charset=utf-8
```

```
{
  "id": 1,
  "running": false,
  "success": true,
  "exitCode": 0,
  "error": null,
  "started": 1537227661847,
  "ended": 1537227672413
}
```

Upload Data: **POST** /api/projects/*name*/upload

Upload data for the named project. Implements the task API.

Accepts the following arguments in a **multipart/form-data** encoded HTTP request body:

- *mesh*: mesh data file (required); data format will be implied from filename extension
- *texture*: texture image file (optional); data format will be implied from filename extension
- *tileid*: tile ID string; must be given if and only if the tiling scheme for the project is **UserDefined** (this argument may also be specified as a URL parameter).

Fails with HTTP status 400 (bad request) if the named project does not exist.

If the project is deleted or run quickly after a call to this API then it is possible that the HTTP status will be 200 (OK) but the upload will not ultimately succeed.

Data formats are implied from the filename extension.

Supported extensions for mesh files include **.obj** and **.ply**. PLY files may be ASCII or binary.

Supported extensions for image files include **.tiff**, **.tif**, **.jpg**, and **.png**. TIFF files may be 8 or 16 bit. Images may be greyscale or 3 band color.

If more than upload is made with the same mesh basename (filename excluding extension) in a project then only the most recently uploaded data will be used.

All uploads must be complete before a project starts running. The inputs used by a project are determined at the time the project is first run. Additional inputs uploaded to the project after that time are ignored, even if the project is re-run.

Input data is validated when a project is run, not on upload.

Example: upload “inputMeshSmall.ply” and “inputImage.png” to “testproj”

Request

```
POST https://landform.hi.jpl.nasa.gov/api/projects/testproj/upload HTTP/1.1
Host: https://landform.hi.jpl.nasa.gov
x-landform-token: API_TOKEN
Content-Type: multipart/form-data; boundary=----WebKitFormBoundary7MA4YWxkTrZu0gW

-----WebKitFormBoundary7MA4YWxkTrZu0gW
Content-Disposition: form-data; name="mesh"; filename="inputMeshSmall.ply"
Content-Type:

DATA
-----WebKitFormBoundary7MA4YWxkTrZu0gW
Content-Disposition: form-data; name="texture"; filename="inputImage.png"
Content-Type: image/png
```

```

DATA
-----WebKitFormBoundary7MA4YWxkTrZuOgW--

Response

HTTP/1.1 200
status: 200
content-type: application/json; charset=utf-8

{
  "id": 2,
  "running": false,
  "success": true,
  "exitCode": 0,
  "error": null,
  "started": 1537228976377,
  "ended": 1537228980192
}

```

Run Project: POST /api/projects/*name*/run

Initiate a run of a project. Implements the task API.

Fails with HTTP status 400 (bad request) if the named project does not exist.

If the project is deleted quickly after a call to this API then it is possible that the HTTP status will be 200 (OK) but the run will fail.

This task only initiates the execution of a project. To determine whether the project execution has completed, do one of the following:

- poll the project metadata via /api/projects/**name** and wait for `Project.FinishedRunning=true`
- get the project result URL via /api/projects/**name**/result?redirect=false and poll it until the status is HTTP 200 (OK).

It is safe to issue this command more than once. However, project results are only computed once: in the absence of errors, subsequent runs of a project after the first run has begun have no effect.

Example: run project “testproj”

Request

```

POST /api/projects/testproj/run HTTP/1.1
Host: https://landform.hi.jpl.nasa.gov
x-landform-token: API_TOKEN
Content-Type: multipart/form-data; boundary=----WebKitFormBoundary7MA4YWxkTrZuOgW

-----WebKitFormBoundary7MA4YWxkTrZuOgW--

```

Response

HTTP/1.1 200

status: 200

content-type: application/json; charset=utf-8

```
{
  "id": 3,
  "running": false,
  "success": true,
  "exitCode": 0,
  "error": null,
  "started": 1537229779144,
  "ended": 1537229780231
}
```

Get Project Metadata: GET /api/projects/*name*

Get JSON metadata for a project.

Fails with HTTP status 400 (bad request) if the named project does not exist.

The project metadata is returned as a JSON object with at least the following fields:

- **Project**
 - **Name**: project name
 - **TilingScheme**, **SkirtMode**, **FacesPerTile**, **TileResolution**: correspond to the options when the object was created
 - **StartedRunning**: whether the project has started execution
 - **FinishedRunning**: whether the project has finished execution
- **Inputs**: array of project inputs
 - **Name**: name of this input
 - **MeshUrl**: URL of the mesh for this input
 - **ImageUrl**: URL of the image for this input
 - **Processed**: whether this input has been processed yet
 - **ImageBands**, **ImageWidth**, **ImageHeight**: image metadata, null if the input has not yet been processed
- **NumNodes**: the total number of 3DTiles hierarchy nodes defined for the project, or null if that has not been computed yet
- **NumProcesedNodes**: number of hierarchy nodes processed so far, or null if the project has not yet begun execution
- **OutputUrl**: the URL at which the final `tileset.json` is expected, see `/api/projects/*name*/result` for more info

Example: get metadata for project “testproj”

Request

```
GET /api/projects/testproj HTTP/1.1
Host: https://landform.hi.jpl.nasa.gov
x-landform-token: API_TOKEN
```

Response

```
HTTP/1.1 200
status: 200
content-type: application/json; charset=utf-8
```

```
{
  "Project": {
    "Name": "testproj",
    "TilingScheme": "Bin",
    "SkirtMode": "None",
    "ReconMethod": "Poisson",
    "FacesPerTile": 2000,
    "TileResolution": 256,
    "TilesDefined": true,
    "ProjectType": "GenericTiling",
    "StartedRunning": true,
    "FinishedRunning": true
  },
  "Inputs": [
    {
      "Name": "inputMeshSmall",
      "MeshUrl": "https://landlords-dev.s3.amazonaws.com/landformweb/input/testproj/inputMeshSmall",
      "ImageUrl": "https://landlords-dev.s3.amazonaws.com/landformweb/input/testproj/inputMeshSmall",
      "Processed": true,
      "ImageBands": 3,
      "ImageWidth": 4096,
      "ImageHeight": 4096
    }
  ],
  "NumNodes": 111,
  "NumProcessedNodes": 111,
  "OutputUrl": "https://landlords-dev.s3.amazonaws.com/landformweb/www/testproj/tileset.json"
}
```

List Projects: GET /api/projects

Get a list of the existing project names.

The project names are returned as a JSON array of strings.

Example:

Request

```
GET /api/projects HTTP/1.1
Host: https://landform.hi.jpl.nasa.gov
x-landform-token: API_TOKEN

Response

HTTP/1.1 200
status: 200
content-type: application/json; charset=utf-8

[
  "testproj",
  "testproj1"
]
```

Get Project Result URL: GET /api/projects/*name*/result

Fetch 3DTiles (<https://github.com/AnalyticalGraphicsInc/3d-tiles>) `tileset.json` result for a project.

By default this API will return a HTTP 302 (found) redirect to the `tileset.json` file stored on AWS. If execution of the project is not yet completed then AWS will return HTTP 403 (forbidden).

If the argument `redirect=false` is specified then the output is the URL to the `tileset` with no redirect.

Example: get result URL for project “testproj” without redirect

Request

```
GET /api/projects/testproj/result?redirect=false
Host: https://landform.hi.jpl.nasa.gov
x-landform-token: API_TOKEN
```

Response

```
HTTP/1.1 200
status: 200
content-type: application/json; charset=utf-8
```

```
https://landlords-dev.s3.amazonaws.com/landformweb/www/testproj/tileset.json
```

View Project: GET /api/projects/*name*/view

Launch a web-based 3D viewer for a completed project.

By default this API will return a HTTP 302 (found) redirect to a web-based viewer for the project.

If the argument `redirect=false` is specified then the output is the viewer URL with no redirect.

Example: launch 3D viewer for project “testproj”

Request

```
GET /api/projects/testproj/view HTTP/1.1
Host: https://landform.hi.jpl.nasa.gov
x-landform-token: API_TOKEN
```

Response

```
HTTP/1.1 302 Found
location: /viewer/index.html?TilesetURL=https%3A%2F%2Flandlords-dev.s3.amazonaws.com%2Flandl
content-type: text/html; charset=utf-8
```

Delete Project: DELETE /api/projects/*name*

Delete the named project. Implements the task API.

Fails with HTTP status 400 (bad request) if the named project does not exist or is currently running.

If multiple calls to this API are made in rapid succession with the same project name only one will succeed. However, in this case it is possible that the HTTP status of more than one of the calls may be HTTP 200 (OK).

If the project is run quickly after this API is called then the deletion may fail even though the status of the delete call may have been HTTP 200.

Example: delete project “testproj”

Request

```
DELETE /api/projects/testproj HTTP/1.1
Host: https://landform.hi.jpl.nasa.gov
x-landform-token: API_TOKEN
```

Response

```
HTTP/1.1 200
status: 200
content-type: application/json; charset=utf-8
```

```
{
  "id": 1,
  "running": false,
  "success": true,
  "exitCode": 0,
  "error": null,
  "started": 1537227661847,
  "ended": 1537227672413
}
```

Task API

Many API calls launch a task. By default such calls are synchronous in that they will not return a response until the task completes. At that point an `application/json` response will normally be sent with the following task metadata:

```
{
  id: server assigned unique task identifier
  name: server assigned task name (may not be unique)
  running: false
  success: whether task succeeded (true or false)
  exitCode: 0 on success, nonzero integer or string error code on failure
  error: error message if any
  started: task start time (ms since epoch on server)
  ended: task end time (ms since epoch on server)
}
```

The HTTP status will be 200 (ok) if the task completed successfully or 500 (server error) if it failed.

If the server encountered an error processing the request (e.g. invalid API call) or failed to launch the task it will return an `application/json` failure response containing only the following error metadata:

```
{
  success: false
  error: error message if any
}
```

In this case the HTTP status will be 400 (bad request) if the API call was invalid or 500 if the task failed to launch for some other reason.

If the task was successfully launched (i.e. if a task `id` was returned):

- The text output of the task may be retrieved via the `/api/tasks/id/log` API.
- The task metadata may be retrieved again via the `/api/tasks/id` API.

The server will expire task metadata and logs after 24h has expired since the completion of the task, after which they will no longer be available.

Asynchronous Task API If the caller prefers an asynchronous task interface the request may include the optional argument `async=true`. In this case the server will respond with the task (or API error) metadata without waiting for the task to complete. The HTTP status will be 200 if the task launched successfully, 400 if the API call was invalid, or 500 if the task failed to launch.

If the task was successfully launched the caller may monitor execution of the task by subsequently polling the `/api/tasks/id` API. The `success`, `exitCode`, `error`,

and `ended` fields are only valid once the task is completed, i.e. `running=false`.

Text Task API If the caller prefers to receive the text output of the task instead of the task metadata the request may include the optional argument `text=true` or `live=true` (the latter takes precedence over the former, and `async=true` takes precedence over both).

When `text=true` the server returns the text output of the task as `text/plain` synchronously, i.e. all at once when the task completes.

When `live=true` the server returns the text output of the task as `text/plain` progressively, i.e. line-by-line as it is produced.

In either case

- If the server failed to launch the task it will return API error metadata as `application/json` and an HTTP status code of 400 or 500.
- If the task was successfully launched the first line of output will give the task ID.
- If the task fails an error message will be appended to the text output.

If the task was successfully launched

- For `text=true` the HTTP status will be 200 or 500 depending on whether the task completed successfully.
- For `live=true` the HTTP status will be 200 whether or not the task completed successfully (because the status code may need to be sent before the task completes).

Get Master Task ID: GET /api/tasks/master/id

Returns id of the master task.

Example:

Request

```
GET /api/tasks/master/id HTTP/1.1
Host: SERVER_URL
x-landform-token: API_TOKEN
```

Response

```
HTTP/1.1 200
status: 200
content-type: application/json; charset=utf-8
```

0

Get Task Metadata: GET /api/tasks/*id*

Returns JSON metadata for task with the given id.

Example: get metadata for task 0 (typically task 0 is the master task)

Request

```
GET /api/tasks/0 HTTP/1.1
Host: https://landform.hi.jpl.nasa.gov
x-landform-token: API_TOKEN
```

Response

```
HTTP/1.1 200
status: 200
content-type: application/json; charset=utf-8
```

```
{
  "id": 0,
  "running": true,
  "success": false,
  "exitCode": null,
  "error": null,
  "started": 1537227621242,
  "ended": null
}
```

Get Task Log: GET /api/tasks/*id*/log

Returns the log for task with the given id.

By default the log is returned as an JSON array of strings containing the line-by-line text output of the task. If the task is still running then whatever it has output up to the time of call will be returned.

If the argument `text=true` is specified then the output is `text/plain` lines instead of a JSON array.

If the argument `live=true` is specified then the output is `text/plain` lines starting with the log to-date. If the task is still running then subsequent lines will be sent progressively until the task is complete.

Example: get text log of task 0 (typically task 0 is the master task)

Request

```
GET /api/tasks/0/log?text=true HTTP/1.1
Host: https://landform.hi.jpl.nasa.gov
x-landform-token: API_TOKEN
```

Response

HTTP/1.1 200
status: 200
content-type: text/plain; charset=utf-8

logging to C:\cygwin64\home\vona\src\mine\Landform\TilingServer\bin\Release\log\log-TilingSe
logging to C:\cygwin64\home\vona\src\mine\Landform\TilingServer\bin\Release\log\log-TilingSe
2020-03-06 10:31:34,230 master INFO: Architecture: x64
2020-03-06 10:31:34,241 master INFO: Venue: landform-dev-vona-quarth
2020-03-06 10:31:34,241 master INFO: Storage URL: s3://landlords-dev/landform-web
2020-03-06 10:31:34,241 master INFO: using 36 of 36 CPU cores
2020-03-06 10:31:34,241 master INFO: LRU image cache capacity 100, LRU data product cache ca
2020-03-06 10:31:34,241 master INFO: AWS profile: landlords
2020-03-06 10:31:34,241 master INFO: AWS region: us-west-1
2020-03-06 10:31:34,241 master INFO: MSLICE AWS profile: mslice
2020-03-06 10:31:34,241 master INFO: MSLICE AWS region: us-west-1
2020-03-06 10:31:34,241 master INFO: MSLICE S3 URL: s3://red-product
2020-03-06 10:31:34,445 master INFO: initialize database
2020-03-06 10:31:34,447 master INFO: initializing 4 database tables for tiling...
2020-03-06 10:31:34,771 master INFO: table "landform-dev-vona-quarth-TilingProjects" exists,
2020-03-06 10:31:34,789 master INFO: table "landform-dev-vona-quarth-TilingInput" exists, 1
2020-03-06 10:31:34,804 master INFO: table "landform-dev-vona-quarth-TilingNode" exists, 49
2020-03-06 10:31:34,822 master INFO: table "landform-dev-vona-quarth-TilingInputChunk" exist
2020-03-06 10:31:34,898 master INFO: initialized 4 database tables
2020-03-06 10:31:34,899 master INFO: initialize database: 0.453s, total 0.453s
2020-03-06 10:31:34,899 master INFO: initialize message queues
2020-03-06 10:31:35,294 master INFO: queue "landform-dev-vona-quarth-tiling-master" exists,
2020-03-06 10:31:35,338 master INFO: queue "landform-dev-vona-quarth-tiling-worker" exists,
2020-03-06 10:31:35,338 master INFO: queues initialized
2020-03-06 10:31:35,338 master INFO: initialize message queues: 0.439s, total 0.439s
2020-03-06 10:31:35,340 master INFO: Architecture: x64
2020-03-06 10:31:35,340 master INFO: Venue: landform-dev-vona-quarth
2020-03-06 10:31:35,340 master INFO: Storage URL: s3://landlords-dev/landform-web
2020-03-06 10:31:35,340 master INFO: using 36 of 36 CPU cores
2020-03-06 10:31:35,340 master INFO: LRU image cache capacity 100, LRU data product cache ca
2020-03-06 10:31:35,340 master INFO: AWS profile: landlords
2020-03-06 10:31:35,340 master INFO: AWS region: us-west-1
2020-03-06 10:31:35,340 master INFO: MSLICE AWS profile: mslice
2020-03-06 10:31:35,340 master INFO: MSLICE AWS region: us-west-1
2020-03-06 10:31:35,341 master INFO: MSLICE S3 URL: s3://red-product